

CODICI

```
// pre: 0 <= n == dim di B[]
// pre: 0 <= n == dim di B[]
// post: in B[0..n-1] i valori verde precedono i bianco ed i rosso, i bianco
precedono i rosso
void Bandiera(Color B[], int n)
{
    int i = 0;
    int j = 0;
    // inv. B[0..i-1] sono verde, B[i..j-1] bianco e B[j..k-1] rosso
    for(int k = 0; k < n; k++) {
        if(B[k] == bianco) {
            swap(B, j, k);
            j++;
        }
        else if(B[k] == verde) {
            swap(B, j, k);
            swap(B, i, j);
            j++;
            i++;
        } // altrimenti B[k] == rosso dunque basta incrementare k ad opera del for
    }
}

int countBubbleSort(int A[], int n) {
    int comparisons = 0;
    int swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = 0; // Flag per verificare
        for (int j = 0; j < n - i - 1; j++) {
            comparisons++;
            if (A[j] > A[j + 1]) {
                swap(A, j, j + 1);
                swapped = 1;
            }
        }
        if (swapped == 0) break;
    }
    return comparisons;
}
```

```

int Partition(int A[], int p, int r) {
    int x = A[p];
    int i = p + 1;
    int j = r;
    while (i <= j) {
        if (A[i] <= x) i = i + 1;
        else if (A[j] > x) j = j - 1;
        else // A[i] > x && A[j] <= x
        {
            swap(A, i, j);
            i = i + 1;
            j = j - 1;
        }
    }
    int q = i - 1;
    swap(A, p, q);
    return q;
}

```

```

list merge(list l, list m) {
    if (l == NULL) {
        return m;
    }
    else if (m == NULL) {
        return l;
    }
    else if (l->info <= m->info) {
        l->next = merge(l->next, m);
        return l;
    }
    else { // l->info > m->info
        m->next = merge(l, m->next);
        return m;
    }
}

```

```

int binomialRec(int n, int k){
    if(k == 0 || k == n){
        return 1;
    }
    else{
        return binomialRec(n-1,k-1) + binomialRec(n-1,k);
    }
}

int binomialBottUp(int n, int k){
    int C[n + 1][k + 1];
    int i;
    int j;
    for (i = 0; i <= n; i++){
        for (j = 0; j <= min(i, k); j++){
            if (j == 0 || j == i){
                C[i][j] = 1;
            }
            else{
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
            }
        }
    }
    return C[n][k];
}

int binomialNoMem(int n, int k) {
    int n_k = 1;
    for (int i = 1; i <= k; i++)
        n_k = (n_k * (n - i + 1))/i;
    return n_k;
}

```

```

Matrix distance(string X, string Y) {
    int m = strlen(X);
    int n = strlen(Y);
    Matrix D = newMatrix(m + 1, n + 1);
    for(int i = 0; i <= m; i++) {
        D->array[i][0] = i;
    }
    for(int j = 0; j <= n; j++) {
        D->array[0][j] = j;
    }
    for(int i = 1; i <= m; i++) {
        for(int j = 1; j <= n; j++){
            if (X[i-1] == Y[j-1])
                D->array[i][j] = D->array[i-1][j-1];
            else
                D->array[i][j] = 1 + min (D->array[i][j-1], min(D->array[i-1][j], D-
>array[i-1][j-1]));
        }
    }
    return D;
}

```

```

list deleteAll(int n, list as) {
    if (as == NULL)
        return NULL;
    else if (as->info == n) {
        list t = as->next;
        free(as);
        return deleteAll(n, t);
    }
    else {
        as->next = deleteAll(n, as->next);
        return as;
    }
}

```

```

list deleteAll(int n, list as) {
    if (as == NULL)
        return NULL;
    else if (as->info == n) {
        list t = as->next;
        free(as);
        return deleteAll(n, t);
    }
    else {
        as->next = deleteAll(n, as->next);
        return as;
    }
}

```

```

list insert(list as, list bs, int n) {
    if (n == 0)
        return concat(as, bs);
    else {
        bs->next = insert(as, bs->next, n - 1);
        return bs;
    }
}

int corank_aux(list l, int cumulative_corank) {
    if (l == NULL) return cumulative_corank;
    int tmp = l->info;
    l->info += cumulative_corank;
    return corank_aux(l->next, cumulative_corank + tmp);
}

int corank(list l) {
    return corank_aux(l, 0);
}

list fast_reverse_aux(list l, list m) {
    if (l == NULL)
        return m;
    else
        return fast_reverse_aux(l->next, Cons(l->info, m));
}

list fast_reverse(list l) {
    return fast_reverse_aux(l, NULL);
}

int height(kTree t) {
    if (t->child == NULL)
        return 0;
    else {
        int ht = 0;
        kTree c = t->child;
        while (c != NULL) {
            ht = max(ht, height(c));
            c = c->sibling;
        }
        return ht + 1;
    }
}

```

```

int sumLeaf (kTree t) {
    if (t == NULL) return 0;
    else if (t->child == NULL) // t è una foglia
        return t->key;
    else {
        kTree c = t->child;
        int sum = 0;
        while (c != NULL) {
            sum = sum + sumLeaf(c);
            c = c->sibling;
        }
        return sum;
    }
}

list kTreeBFS(kTree t) {
    if (t == NULL)
        return NULL;
    list l = NULL;
    queue q = NewQueue();
    EnQueue(t, q);
    while(!isEmptyQueue(q)) {
        kTree node = DeQueue(q);
        l = Cons(node->key, l); // visita
        // l = Add(node->key, l); // versione con Add
        node = node->child;
        while (node != NULL) {
            EnQueue(node, q);
            node = node->sibling;
        }
    }
    return reverse(l);
    // return l; // versione con Add
}

```

```

Triple isOrderedBTreeAux(btree bt)
{
    Triple t;
    if(isLeaf(bt)) // true se bt->left == NULL && bt->right == NULL
    {
        t.isOrdered = true;
        t.min = t.max = bt->key;
    }
    else if(bt->right == NULL) // quindi bt->left != NULL
    {
        Triple tleft = isOrderedBTreeAux(bt->left);
        t.isOrdered = tleft.isOrdered && bt->key > tleft.max;
        t.max = bt->key;
        t.min = tleft.min;
    }
    else if(bt->left == NULL) // quindi bt->right != NULL
    {
        Triple tright = isOrderedBTreeAux(bt->right);
        t.isOrdered = tright.isOrdered && bt->key < tright.min;
        t.max = tright.max;
        t.min = bt->key;
    }
    else // bt->left != NULL && bt->right != NULL
    {
        Triple tleft = isOrderedBTreeAux(bt->left);
        Triple tright = isOrderedBTreeAux(bt->right);
        t.isOrdered = tleft.isOrdered &&
                        tright.isOrdered &&
                        bt->key > tleft.max && bt->key < tright.min;
        t.min = tleft.min;
        t.max = tright.max;
    }
    return t;
}

```

```

bool isOrdered(btree bt)
{
    if (bt == NULL)
        return true;
    else
    {
        Triple t = isOrderedBTreeAux(bt);
        return t.isOrdered;
    }
}

```

```

list DescList_aux(btree bt, list l) {
    if (bt == NULL)
        return l;
    else {
        l = DescList_aux(bt->left, l);
        l = Cons(bt->key, l);
        return DescList_aux(bt->right, l);
    }
}

list DescList(btree bt) {
    return DescList_aux(bt, NULL);
}

btree minInBtree(btree bt) {
    if(bt->left == NULL)
        return bt;
    else
        return minInBtree(bt->left);
}

btree rightAncestor(btree nd) {
    if(nd->parent == NULL)
        return NULL;
    else if(nd == nd->parent->left)
        return nd->parent;
    else
        return rightAncestor(nd->parent);
}

btree successor(btree nd) {
    if(nd->right != NULL)
        return minInBtree(nd->right);
    else
        return rightAncestor(nd);
}

```



```

void buildMaxHeap(int A[], int dim) {
    int m = dim / 2 + 1;
    for (int i = m; i >= 0; i--) maxHeapify(A, i, dim);
}
void heapSort(int A[], int dim) {
    buildMaxHeap(A, dim - 1);
    for (int i = dim - 1; i > 0; i--) {
        swap(A, 0, i);
        maxHeapify(A, 0, i - 1);
    }
}

```

```

int find(int A[], int N, int f) {
    int m = 0, n = N-1;
    while(m < n) {
        int i = m, j = n, r = A[f];
        while(i <= j) {
            while(A[i] < r) i += 1;
            while(r < A[j]) j -= 1;
            if(i <= j) {
                swap(A, i, j);
                i += 1, j -= 1;
            }
        }
        if(f <= j) n = j;
        else if(i <= f) m = i;
        else break;
    }
    return A[f];
}

```

```

int cardinality(btree t) {
    if (t == NULL) return 0;
    else {
        int l = cardinality(t->left);
        int r = cardinality(t->right);
        return l + r+1;
    }
}

```

```
}
```

```
struct HeapFrame {  
    int dim, hd;  
    int* keys;  
};  
  
typedef struct HeapFrame* Heap;  
  
int parent(Heap H, int i) {return i / 2;}  
  
bool insert(Heap H, int k) {  
    if (H == NULL) return false;  
    int dim = H->dim;  
    int i = H->hd;  
    int* keys = H->keys;  
    // in caso di allocazione vedere sotto  
    keys[i] = k;  
    H->hd++;  
    while (i > 0 && keys[parent(H, i)] > keys[i]) {  
        swap(keys, i, parent(H, i));  
        i = parent(H, i);  
    }  
    return true;  
}
```

```

// riallocazione

// if (i > dim) {

//     // Riallocazione dell'heap se necessario
//     int newDim = dim * 2;
//     int* newKeys = (int*)realloc(keys, newDim * sizeof(int));
//     if (newKeys == NULL) {
//         // Gestione dell'errore di allocazione
//         printf("Errore: impossibile allocare memoria\n");
//         return false;
//     }
//     H->keys = newKeys;
//     H->dim = newDim;
// }

void DFS(kTree t){
    // visita t.key
while(t->child != NULL){

    DFS(t->child);

    t->child=t->child->sibling;

}
}

int height(kTree t){

    if(t==NULL) return 0;

    if(t->child==NULL)return 0;

    int sum = 0;

    while(t->child!=NULL){

```

```

        sum = max(sum,height(t->child));

        t->child=t->child->sibling;

    }

    return sum+1;
}

```

```

int sumLeaf(kTree t) {

    if(t==NULL) return 0;

    if(t->child==NULL) return t->key;


    int sum = 0;

    while(t->child!=NULL) {

        sum += sumLeaf(t->child);

        t->child = t->child->sibling;

    }

    return sum;

}

```

```

int cardinality(kTree t) {

    if (t == NULL) return 0;
    else {

        int card=1;

        kTree child = t->child;

        while(child != NULL){

            card += cardinality(child);

            child = child->sibling;

        }

    }
}

```

```

        return card;

    }

}

int degree(kTree vertex) {

    if (vertex == NULL) return 0;

    int num_degree = 0;

    kTree child = vertex->child;

    while (child != NULL) {

        int childDegree = degree(child);

        num_degree = (childDegree > num_degree) ?
            childDegree : num_degree;

        child = child->sibling;

    }

    return num_degree + 1;

}

int Shortest(kTree t) {

    if (t == NULL) return 0;

    if (t->child == NULL) return 1;

    int shortest = 50000000;

    while (t->child != NULL) {

        int pathLength = Shortest(t->child);

        if (pathLength < shortest) {

            shortest = pathLength;

        }

    }

}

```

```

        t->child = t->child->sibling;

    }

    return shortest + 1; // Aggiungiamo 1 per includere la radice
}

// implemented with bfs from left to right
int Large(kTree t) {
    queue q = NewQueue();
    EnQueue(t, q);
    int count = 0;
    while(!isEmptyQueue(q)) {
        kTree current = DeQueue(q);
        int max_node = 0;
        while(current->child != NULL) {
            EnQueue(current->child, q);
            max_node = max_node+1;
            current=current->child;
        }
        if(max_node>count) count = max_node;
    }
    return count+1;
}

// implemented without bfs
int LargeNOBFS(kTree t) {
    if(t==NULL) return 0;

```

```

    if (t->child == NULL) return 1;

    int sum_sib=0;
    int sum_child=0;
    while (t->child!=NULL) {
        sum_sib++;
        sum_child = max(Large(t->child), sum_child);
        t->child= t->child->sibling;
    }
    return max(sum_sib, sum_child);
}

```

```
list kTreeBFS(kTree t) {
```

```

    if (t == NULL) return NULL;

    list l = NULL;

    queue q = NewQueue();
    EnQueue(t, q);

    while (!isEmptyQueue(q)) {
        kTree node = DeQueue(q);

        // l = Cons(node->key, l); // visita
        // l = Add(node->key, l); // versione con Add

        node = node->child;

        while (node != NULL) {
            EnQueue(node, q);
            node = node->sibling;
        }
    }

    // return reverse(l);

```

```

    // return 1; // versione con Add
}

// implemented with bfs
void StampaSommaLivelli(kTree t){
    if (t == NULL) return NULL;

    queue q = NewQueue();
    EnQueue(t, q);
    printf("%d\n", t->key);
    while(!isEmptyQueue(q)) {
        kTree node = DeQueue(q);
        int sum = 0;
        node = node->child;
        while (node != NULL) {
            EnQueue(node, q);
            sum = node->key + sum;
            node = node->sibling;
        }
        printf("%d\n", sum);
    }
}

int countInternalNodes(kTree t){
    if(t==NULL) return 0;
    if(t->child==NULL) return 0;
    int sumInternalNodes = 0;
    while(t->child != NULL){

```



```

        sumInternalNodes += countInt(t->sibling);

        t->child = t->child->sibling;

    }

    return sumInternalNodes+1;

}

kTree complete(int key, int dg, int ht){

    int children =1;

    if(ht == 0){

        return constTree(key, NULL, NULL);

    }

    kTree child = complete( key, dg, ht-1);

    kTree temp = child;

    while(children < dg){

        temp->sibling = complete( key, dg, ht-1);

        children++;

        temp = temp->sibling;

    }

    return constTree(key, child, NULL);

}

int MaxSumBranch(kTree t){

    if(t==NULL) return 0;

    if(t->child==NULL) return t->key;

    int max_sum = MaxSumBranch(t->child);

    kTree c = t->child->sibling;

```

```

while(c != NULL) {

    max_sum = max(max_sum , MaxSumBranch(c));

    c=c->sibling;

}

return t->key+max_sum;

}

```

```

void StampaFiglioMinimo(kTree t){

```

```

    if(t==NULL) return NULL;

    int temp = t->child->key;

    int min = t->child->key;

    while(t->child->sibling != NULL){

        if(t->child!=NULL) {

            StampaFiglioMinimo(t->child->child);

        }

        temp = min(temp, t->child->sibling->key);

    }

    if(t->child->child!=NULL) StampaFiglioMinimo;

    return;

}

```

```

// Aggiunge un nuovo nodo con etichetta uguale alla somma dei suoi
predecessori

```

```

void SumBranchAux(kTree t, int s){

```

```

    if(t->child == NULL){/*crea nodo con etichetta s+t->key*/}

    s= s+t->key;

    kTree child = t->child;

    while(child!=NULL){

        SumBranchAux(child, s);

        child = child->sibling;

    }

}

void SumBranch(kTree t, int s){

    SumBranchAux(t, 0);

}

```

```

void DFS(kTree t){

    // visita t.key
while(t->child != NULL){

    DFS(t->child);

    t->child=t->child->sibling;

}

}

int height(kTree t){

    if(t==NULL) return 0;

    if(t->child==NULL) return 0;

    int sum = 0;

    while(t->child!=NULL){

        sum = max(sum,height(t->child));

    }

}

```

```

        t->child=t->child->sibling;

    }

    return sum+1;
}

int sumLeaf(kTree t) {
    if(t==NULL) return 0;

    if(t->child==NULL) return t->key;

    int sum = 0;

    while(t->child!=NULL) {
        sum += sumLeaf(t->child);

        t->child = t->child->sibling;
    }

    return sum;
}

int cardinality(kTree t) {
    if (t == NULL) return 0;
    else {
        int card=1;

        kTree child = t->child;

        while(child != NULL){
            card += cardinality(child);

            child = child->sibling;
        }

        return card;
    }
}

```

```

    }
}

int degree(kTree vertex) {
    if (vertex == NULL) return 0;

    int num_degree = 0;
    kTree child = vertex->child;
    while (child != NULL) {
        int childDegree = degree(child);
        num_degree = (childDegree > num_degree) ?
            childDegree : num_degree;
        child = child->sibling;
    }
    return num_degree + 1;
}

```

```

int Shortest(kTree t) {
    if (t == NULL) return 0;
    if (t->child == NULL) return 1;
    int shortest = 50000000;
    while (t->child != NULL) {
        int pathLength = Shortest(t->child);
        if (pathLength < shortest) {
            shortest = pathLength;
        }
        t->child = t->child->sibling;
    }
}

```

```

    }

    return shortest + 1; // Aggiungiamo 1 per includere la radice
}

// implemented with bfs from left to right
int Large(kTree t) {
    queue q = NewQueue();
    EnQueue(t, q);
    int count = 0;
    while(!isEmptyQueue(q)) {
        kTree current = DeQueue(q);
        int max_node = 0;
        while(current->child != NULL) {
            EnQueue(current->child, q);
            max_node = max_node+1;
            current=current->child;
        }
        if(max_node>count) count = max_node;
    }
    return count+1;
}

// implemented without bfs
int LargeNOBFS(kTree t){
    if(t==NULL) return 0;
    if(t->child == NULL) return 1;

```

```

int sum_sib=0;

int sum_child=0;

while(t->child!=NULL) {

    sum_sib++;

    sum_child = max(Large(t->child), sum_child);

    t->child= t->child->sibling;

}

return max(sum_sib, sum_child);

}

```

```

list kTreeBFS(kTree t) {

```

```

    if (t == NULL) return NULL;

    list l = NULL;

    queue q = NewQueue();

    EnQueue(t, q);

    while(!isEmptyQueue(q)) {

        kTree node = DeQueue(q);

        // l = Cons(node->key, l); // visita
        // l = Add(node->key, l); // versione con Add

        node = node->child;

        while (node != NULL) {

            EnQueue(node, q);

            node = node->sibling;

        }

    }

    // return reverse(l);

    // return l; // versione con Add

```

```

}

// implemented with bfs
void StampaSommaLivelli(kTree t){

    if (t == NULL) return NULL;

    queue q = NewQueue();

    EnQueue(t, q);

    printf("%d\n", t->key);

    while(!isEmptyQueue(q)) {

        kTree node = DeQueue(q);

        int sum = 0;

        node = node->child;

        while (node != NULL) {

            EnQueue(node, q);

            sum = node->key + sum;

            node = node->sibling;

        }

        printf("%d\n", sum);

    }

}

```

```

int countInternalNodes(kTree t){

    if(t==NULL) return 0;

    if(t->child==NULL) return 0;

    int sumInternalNodes = 0;

    while(t->child != NULL){

        sumInternalNodes += countInt(t->sibling);
    }
}

```



```

        t->child = t->child->sibling;

    }

    return sumInternalNodes+1;

}

kTree complete(int key, int dg, int ht){

    int children =1;

    if(ht == 0){

        return consTree(key, NULL, NULL);

    }

    kTree child = complete( key, dg, ht-1);

    kTree temp = child;

    while(children < dg){

        temp->sibling = complete( key, dg, ht-1);

        children++;

        temp = temp->sibling;

    }

    return consTree(key, child, NULL);

}

int MaxSumBranch(kTree t){

    if(t==NULL) return 0;

    if(t->child==NULL) return t->key;

    int max_sum = MaxSumBranch(t->child);

    kTree c = t->child->sibling;

    while(c != NULL){

```

```

        max_sum = max(max_sum , MaxSumBranch(c));

        c=c->sibling;

    }

    return t->key+max_sum;
}

```

```

void StampaFiglioMinimo(kTree t){

```

```

    if(t==NULL) return NULL;

    int temp = t->child->key;

    int min = t->child->key;

    while(t->child->sibling != NULL){

        if(t->child!=NULL){

            StampaFiglioMinimo(t->child->child);

        }

        temp = min(temp, t->child->sibling->key);

    }

    if(t->child->child!=NULL) StampaFiglioMinimo;

    return;

}

```

// Aggiunge un nuovo nodo con etichetta uguale alla somma dei suoi predecessori

```

void SumBranchAux(kTree t, int s){

    if(t->child == NULL){/*crea nodo con etichetta s+t->key*/}

```

```

    s= s+t->key;

    kTree child = t->child;

    while(child!=NULL) {

        SumBranchAux(child, s);

        child = child->sibling;

    }

}

void SumBranch(kTree t, int s){

    SumBranchAux(t, 0);

}

```

```

// post: fonde due liste distruttivamente

list mergeLists(list xs, list ys) {

    if(xs==NULL) return ys;

    if(ys==NULL) return xs;

    if(xs->info == ys->info){

        return Cons(xs->info, mergeLists(xs->next, ys->next));

    }

    else if(xs->info < ys->info) {

return Cons(xs->info, mergeLists(xs->next, ys));

    }

    else if(xs->info > ys->info) {

return Cons(ys->info, mergeLists(xs, ys->next));

    }

}

// post: fonde due liste NON distruttivamente con copyList()

```

```

list unione(list l, list m) {

    if (m == NULL) return l;

    if (l == NULL) return m;

    list sx = copyList(l);

    list dx = copyList(m);

    list res = NULL;

    if (sx->info < dx->info) {

        res = Cons(sx->info, unione(sx->next, dx));

    } else if (sx->info > dx->info) {

        res = Cons(dx->info, unione(sx, dx->next));

    } else {

        res = Cons(dx->info, unione(sx->next, dx->next));

    }

    return res;

}

// post: inverte distruttivamente l in tempo lineare, ritornando la lista
invertita

list reverseList(list xs){

    list reversed = NULL;

    while (xs != NULL) {

        reversed = Cons(xs->info, reversed);

        xs = xs->next;

    }

    return reversed;

}

// post: genera NON distruttivamente l'inversa di l, davanti a m

```

```
list fast_reverse_aux(list l, list m) {  
    if(l == NULL) return m;  
    else return fast_reverse_aux(l->next, Cons(l->info, m));  
}  
  
// post: inverte NON distruttivamente l in tempo lineare
```

```
list fast_reverse(list l) {
```

```
    return fast_reverse_aux(l, NULL);  
}
```

```
// post: ogni nodo di l contiene il suo rango
```

```
int corank(list l) {  
    if(l==NULL) return 0;  
    if(l->next!=NULL){  
        l->next->info += l->info;  
        return corank(l->next);  
    }else return l->info;  
}
```

```
// post: l è stata trasformata nella lista dei ranghi dei suoi elementi;  
// il valore ritornato è il rango del primo elemento rispetto alla lista  
originaria.
```

```
int rank(list l){  
    if(l==NULL) return 0;  
    if(l->next == NULL) return l->info;  
    int temp = 0;  
    while(l->next!=NULL){
```

```

        temp = rank(l->next);

        l->info=l->info + temp;

        return l->info;

    }

    return l->info;
}

// post: cancella le occorrenze n in as
list deleteAll(int n, list as) {

    if (as == NULL) return NULL;

    else if (as->info == n) {

        list t = as->next;

        free(as);

        return deleteAll(n, t);

    }

    else {

        as->next = deleteAll(n, as->next);

        return as;

    }

}

list symmDiff(list xs, list ys) {

    if (xs == NULL) return copyList(ys);

    else if (ys == NULL) return copyList(xs);

    else if (xs->info == ys->info) return symmDiff(xs->next, ys->next);

    else if (xs->info < ys->info) return Cons(xs->info, symmDiff(xs->next,
ys));

    else return Cons(ys->info, symmDiff(xs, ys->next));
}

```

```

}

// pre: 0 <= n < lunghezza di bs
// post: inserisce distruttivamente as in bs dopo n elementi, seguita
list insert(list as, list bs, int n){
    if(n==0) return concat(as,bs);
    else{
        bs->next = insert(as, bs->next, n-1);
        return bs;
    }
}

```

```

/***** MERGESORT WITH LIST *****/

list merge(list l, list m) {
    if(l==NULL) return m;
    if(m==NULL) return l;

    if (l->info <= m->info) {
        l->next = merge(l->next, m);
        return l;
    }
}

```

```

    }

    else {

        m->next = merge(l, m->next);

        return m;

    }

}

```

```

list Mergesort(list l) {

    if (l == NULL || l->next == NULL)

        return l;

    else {

        list m = split(l); // divide distruttivamente l a metà in modo che
l punti ad una metà, il puntatore ritornato all'altra metà

        l = Mergesort(l);

        m = Mergesort(m);

        return merge(l, m);

    }

}

```

```

list split(list l) {

    if (l == NULL || l->next == NULL)

        return NULL;

    list slow = l;

    list fast = l->next;

    while (fast != NULL) {

        fast = fast->next;
    }
}

```



```

    if (fast != NULL) {
        slow = slow->next;
        fast = fast->next;
    }
}

list second_half = slow->next;

slow->next = NULL;

return second_half;
}

```

```

struct hashFrame {
    int    dim;    // dimensione della tabella
    int* array;    // puntatore alla tabella array[0..dim-1]
};

typedef struct hashFrame* HashTable;

// pre:  0 < m
// post:  ritorna una tabella hash di dimensione m a chiavi
//        positive,
//        inizializzata con tutti -1 che rappresenta l'assenza di
//        chiavi

HashTable newHashTable(int m) {
    HashTable T = malloc(sizeof(struct hashFrame));

    T->dim = m;

    T->array = malloc(m * sizeof(int));

    for(int i = 0; i < m; i++)
        T->array[i] = -1;
}

```

```

        return T;
    }

int hashFun(int k, int m){return k % m;}

int linearProbing(int k, int i, int m){return (hashFun(k, m) + i)
% m;}

// post: j se k è inserito in T->array[j], con LINEAR PROBING
// -1 se k era presente in T
// -2 in caso di overflow

int hashInsert(HashTable T, int k){

    for(int i=0; i<T->dim; i++){

        int pos = linearProbing(k, i, T->dim);

        if(pos >= T->dim) return -2;

        // se è già presente ritorna -1

        if(T->array[pos] == k){

            return -1;

        }

        // inserimento

        if(T->array[pos] == -1){

            T->array[pos] = k;

            return pos;

        }

    }

    return -2;
}

```

HEAP MINIMO

- Riportare le caratteristiche di un *heap minimo*.
- Il seguente array rappresenta un *heap minimo*.

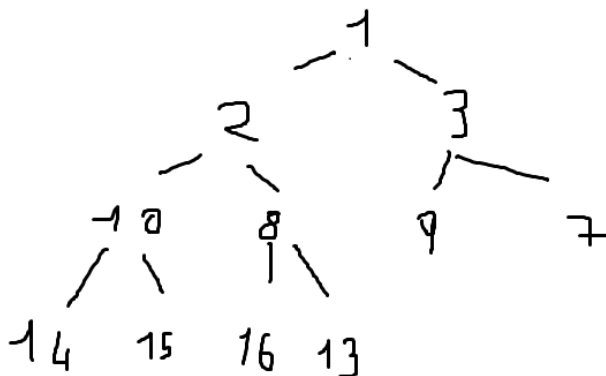
(1, 2, 3, 10, 8, 9, 7, 14, 15, 16, 13)

Effettuare l'estrazione del minimo riportando l'array che rappresenta lo heap dopo ogni scambio di elementi.

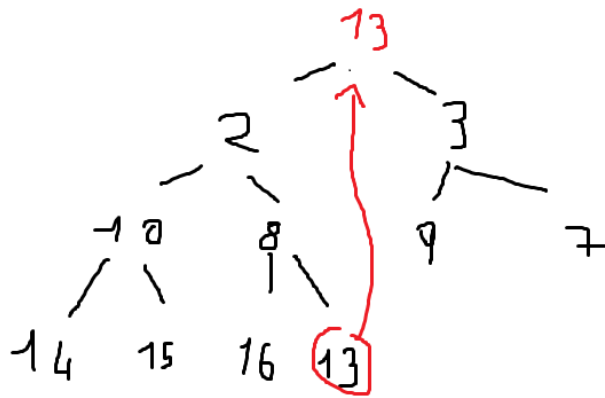
1) Un heap minimo è una struttura dati utilizzata nell'ambito dell'informatica e dell'analisi degli algoritmi. È una forma di albero binario che soddisfa due importanti proprietà:

1. **Proprietà di ordinamento:** Ogni nodo nell'heap ha un valore minore o uguale a quelli dei suoi figli. Questa proprietà garantisce che il nodo radice contenga sempre il valore minimo nell'heap.
2. **Struttura ad albero completo:** L'heap minimo è un albero binario completo, il che significa che tutti i livelli dell'albero sono completamente riempiti, tranne eventualmente l'ultimo, che è riempito da sinistra a destra con nodi.

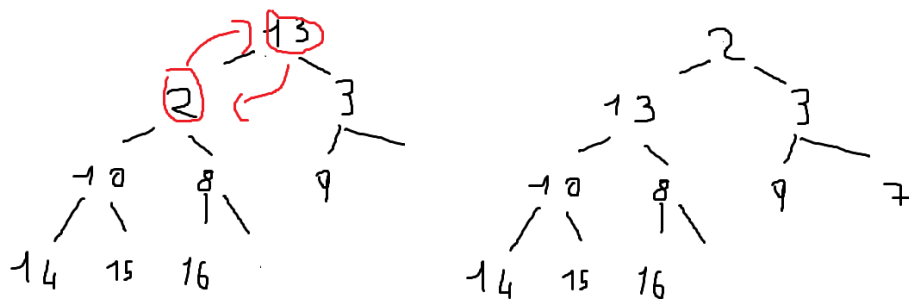
2) prima di tutto ci disegniamo l'albero



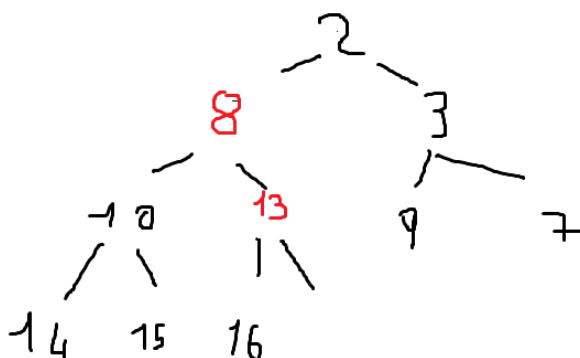
Poi per estrarre il minimo cancelliamo l'uno (che è il minimo), e facciamo delle rotazioni, lo sostituiamo con il numero in basso più a destra (quindi il 13)



Successivamente dobbiamo fare degli spostamenti per far andare il 2 come punto più alto (essendo il minimo)



Infine, siccome la radice deve essere sempre inferiore ai numeri sotto, spostiamo il 13 e l'8 ottenendo:



Ora l'esercizio è finito, bisogna riportare ogni array che rappresenta gli scambi:

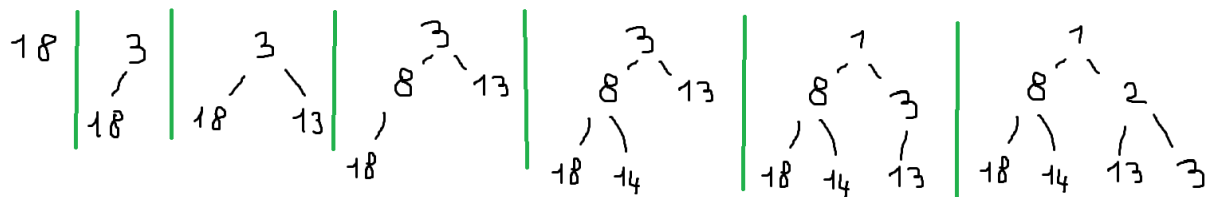
{13,2,3,10,8,9,7,14,15,16}

{2,13,3,10,8,9,7,14,15,16}

{2,8,3,10,13,9,7,14,15,16}

Si inseriscano in un heap minimo inizialmente vuoto le seguenti chiavi: 18, 3, 13, 8, 14, 1, 2.
Si riporti il vettore che rappresenta l'heap dopo ogni inserimento.

Per risolvere questo quesito si inseriscono i valori uno ad uno rispettando il minimo in alto:



Infine, segnare nella risposta i passaggi:

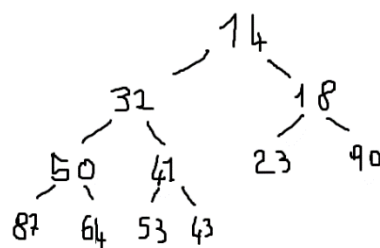
[18], [3,18], [3,18,13], [3,8,13,18], [3,8,13,18,14], [1,8,3,18,14,13], [1,8,2,18,14,13,3]

Si consideri lo heap minimo rappresentato con l'array:

[14, 32, 18, 50, 41, 23, 90, 87, 64, 53, 43]

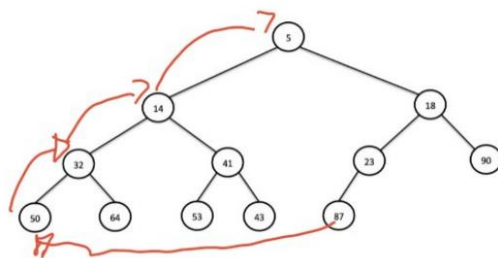
ottenuto dopo l'estrazione del minimo 5, dallo:

- ☐ [5, 14, 18, 32, 41, 23, 90, 50, 64, 53, 43, 87]
 - ☐ [5, 14, 23, 32, 41, 18, 90, 50, 64, 53, 43, 87]
 - ☐ [5, 14, 18, 50, 41, 23, 90, 32, 64, 53, 43, 87]
- Da questo, dopo l'inserimento della chiave 15, si ottiene:
- ☐ [14, 32, 15, 50, 41, 18, 90, 87, 64, 53, 43, 23]
 - ☐ [14, 32, 15, 50, 41, 23, 90, 87, 64, 53, 43, 18]
 - ☐ [14, 50, 15, 32, 41, 18, 90, 87, 64, 53, 43, 23]



Prima di tutto mi disegno l'albero:

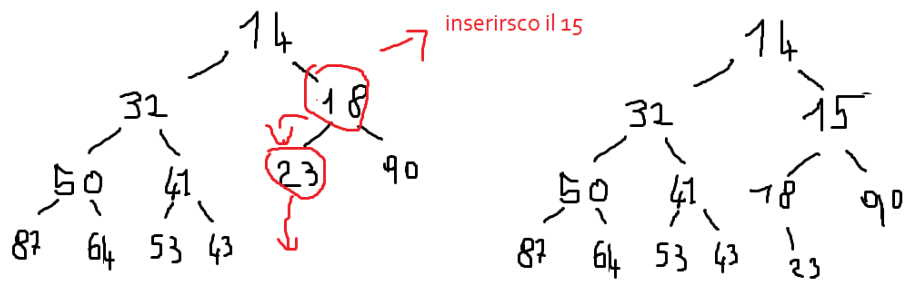
Poi dovremmo immaginare come fosse l'albero se prima avesse avuto un 5, si possono disegnare gli alberi e verificare quali hanno senso, il primo (che è la risposta giusta) è il



seguente:

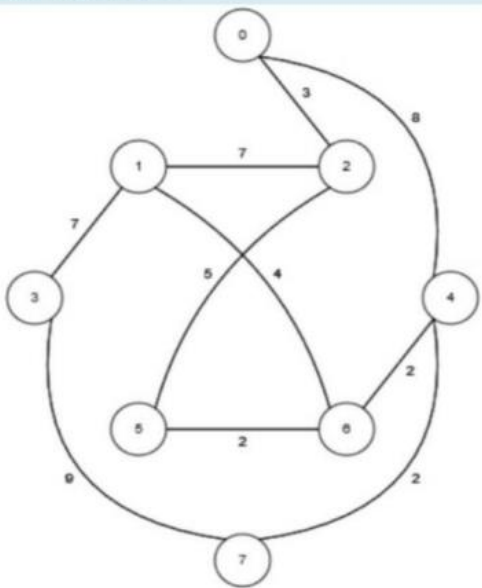
mentre disegnando gli altri possiamo notare subito che non abbiano senso in un heap minimo avendo numeri inferiori sotto una radice.

Invece per inserire la chiave 15 facciamo le operazioni:



GRAFI DI PRIM

Si consideri il seguente grafo:



Si simuli sul grafo l'algoritmo di Prim a partire dal nodo 0. Si riporti il risultato elencando gli archi inclusi nella soluzione.

Per risolvere questo esercizio, dobbiamo ripercorrere dall'inizio i passaggi, considerando il percorso che costa meno:

Prendi il nodo iniziale, guardi dove può andare e vai nel percorso che costa meno. Una volta che hai due nodi, guardi tutti i percorsi disponibili che hai; quindi, da nodo 1 e nodo 2 e da lì guardi quale tra tutti costa meno. E così via.

Nell'esempio

0: disponibili 2 e 4, 2 costa meno, vai in 2

0,2: disponibili 4,7,5 e 1, 5 è quello che costa meno, vai in 5

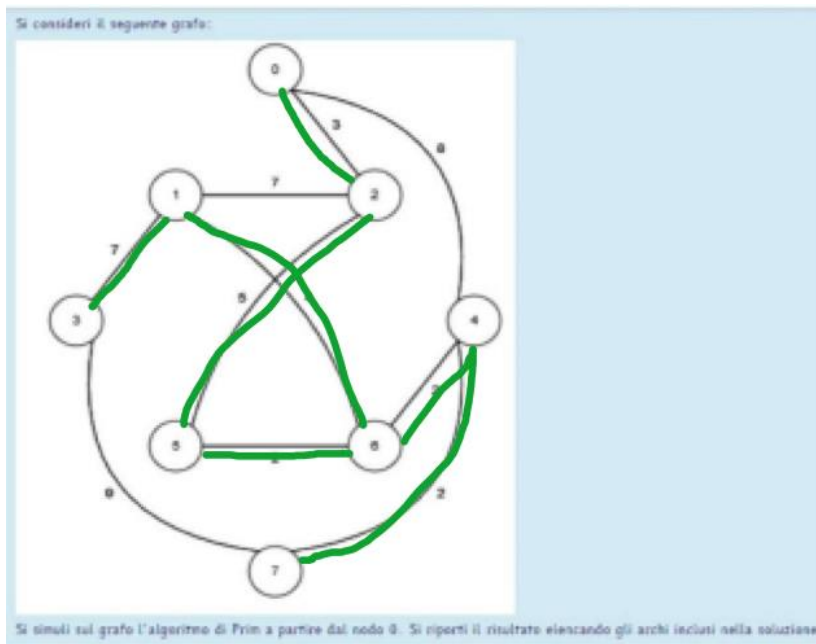
0,2,5: disponibili 6,1 e 4, 6 è quello che costa meno, vai in 6

0,2,5,6: disponibili 4 e 1, 4 passando da 6 è quello che costa meno, vai in 4

0,2,5,6,4: disponibili 7 e 1, 7 costa meno vai in 7

0,2,5,6,4,7: disponibili 3 e 1, 1 passando da 6 è quello che costa meno, vai in 1

0,2,5,6,4,7,1: disponibili 3, vai in tre passando da 1 perché costa meno



Successivamente segnare gli archi come risposta:

0 -> 2 costo 3

2 -> 5 costo 5

5 -> 6 costo 2

6 -> 4 costo 2

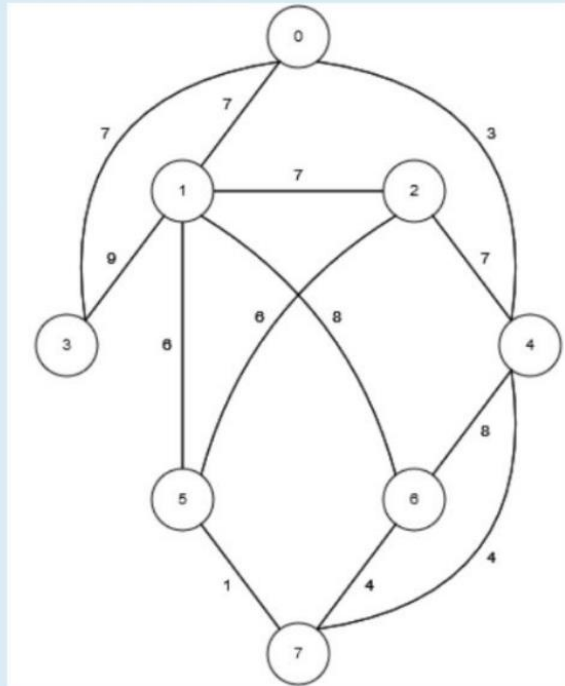
4 -> 7 costo 2

6 -> 1 costo 4

1 -> 3 costo 7

ALGORITMO DI Kruskal

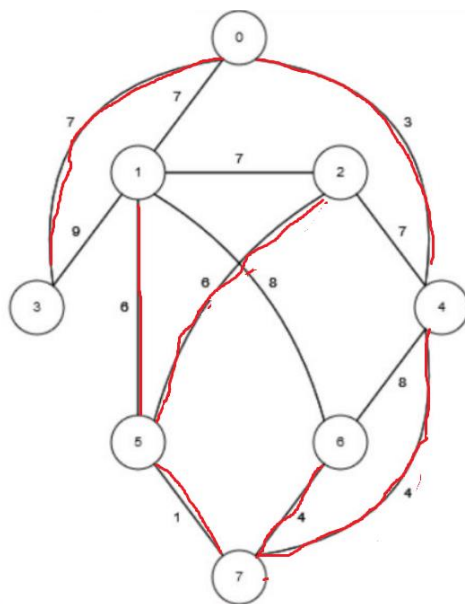
Si consideri il seguente grafo:



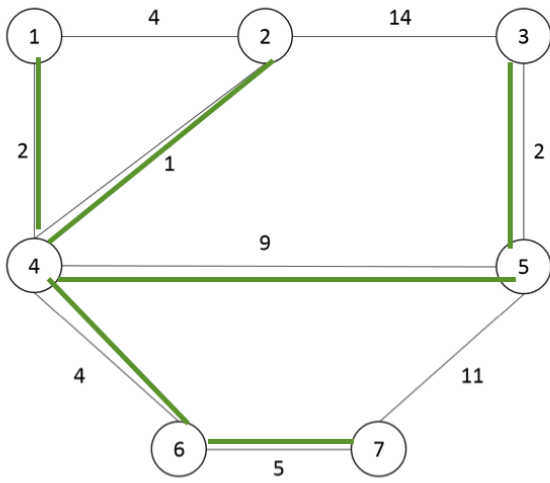
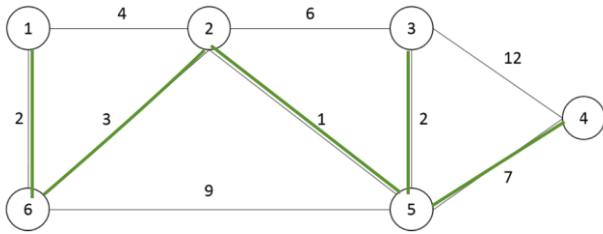
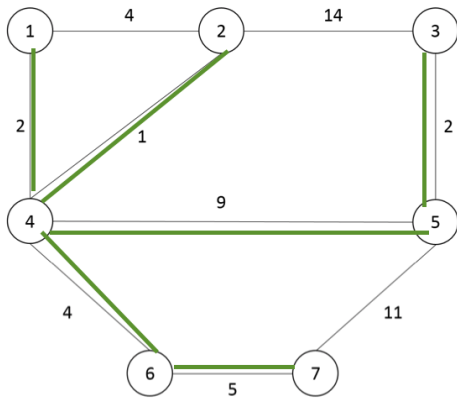
Si simuli sul grafo l'algoritmo di Kruskal. Si riporti il risultato elencando gli archi inclusi nella soluzione.

Per risolvere questo tipo di grafi dobbiamo guardare tutti i percorsi più corti che portano ad un nodo, collegarli stando attenti a non creare dei cicli (ovvero forme chiuse, ad esempio segnano [1,2], [1,5], [5,2] sarebbe un ciclo).

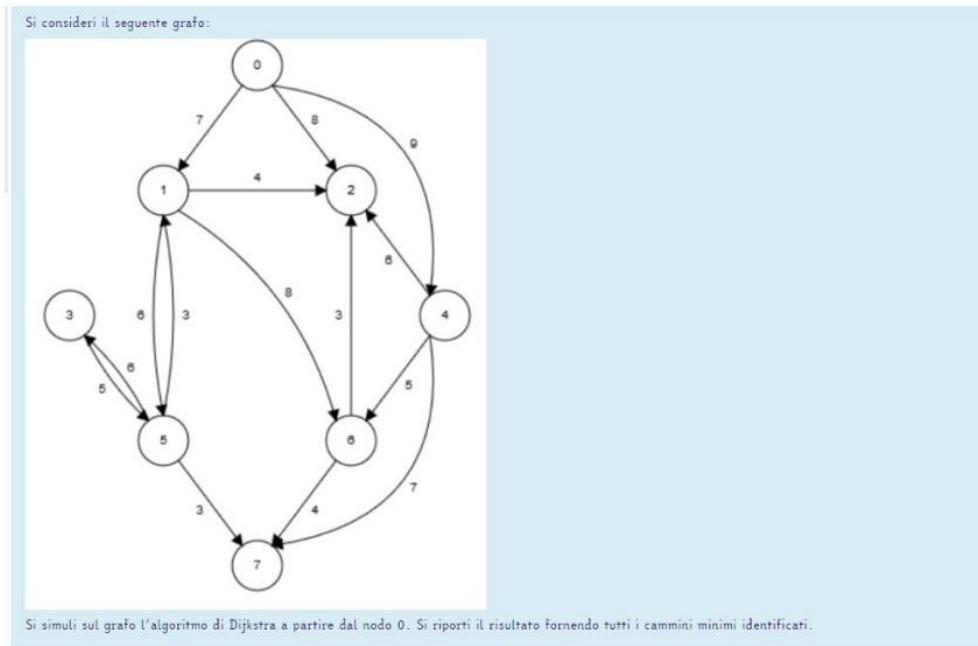
Ad esempio, guardiamo il nodo 2, potrei arrivarci passando dal 4 (costo 7) o dall'1 (costo 7) o ancora dal 5 (costo 6). Quindi scelgo di passarci dal nodo 5. E così via



[7,5][0,4][7,6] [7,4] [5,1] [5,2] [3,0]

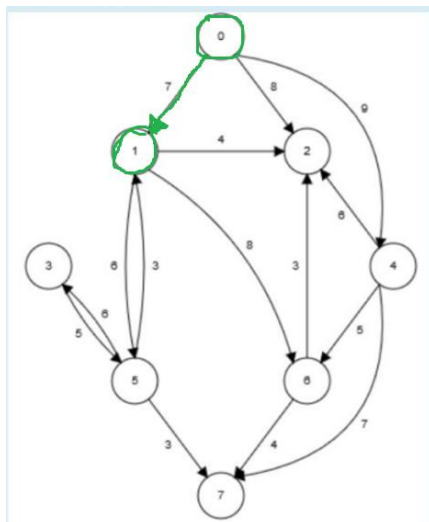


GRAFO DIJKSTRA

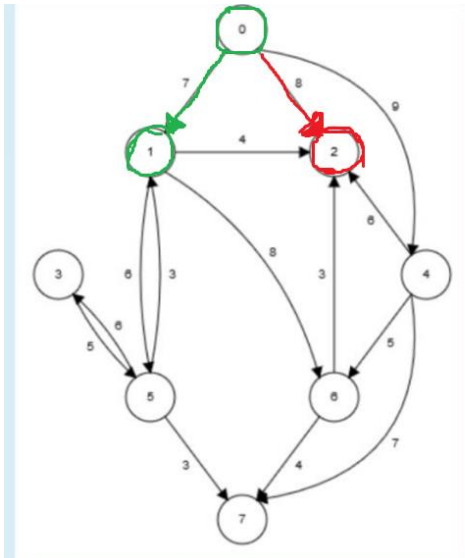


Per risolvere questi grafi dobbiamo trovare i percorsi minimi che consentono di toccare tutti i punti, si può andare solo in maniera sequenziale partendo dal nodo di partenza e guardando le scelte a disposizione per i nodi già visitati. Dobbiamo anche stare attenti alla direzione.

Conviene segnarsi una tabella con tutte le possibili scelte. Guardiamo i passi:



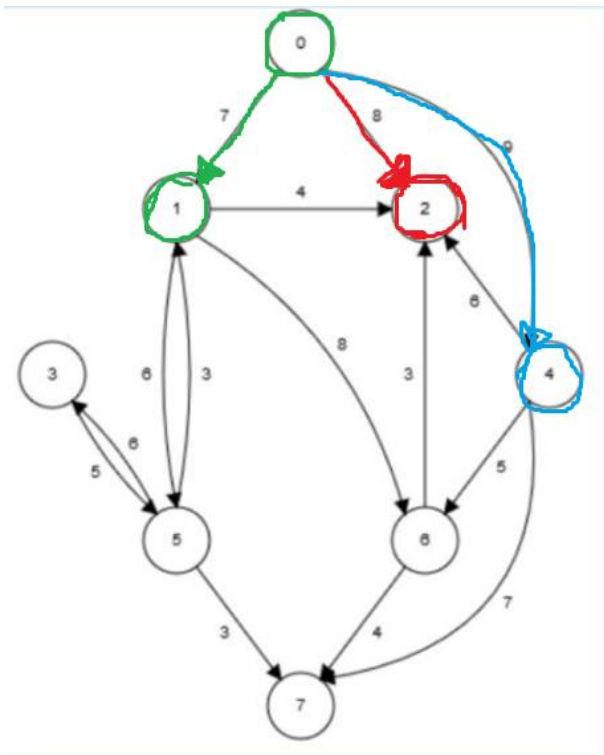
0 = 0 nel primo passo (dobbiamo partire da 0)
1 = 7 segniamo tutti i nodi e i costi relativi, come
2 = 8 possiamo vedere dal nodo 0 possiamo andare
3 = solo al nodo 1, 2 e 4. Scegliamo il nodo 1
4 = 9 perché ha costo inferiore e andiamo avanti.
5 =
6 =
7 =



viaggio dal nodo 0 al nodo 2

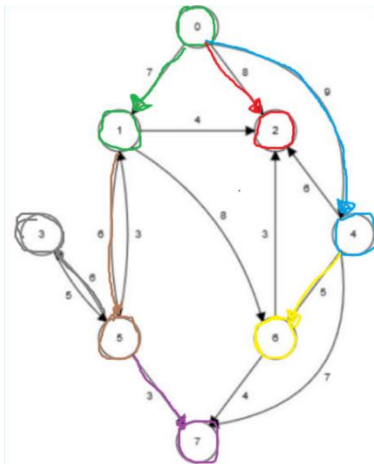
0 = 0	
1 = 7	
2 = 8	2 = 8
3 =	3 =
4 = 9	4 = 9
5 =	5 = 13
6 =	6 = 15
7 =	7 =

al secondo passo scriviamo di nuovo tutti i nodi dove possiamo arrivare (non c'è bisogno di segnare quelli dove siamo già passati), facendo attenzione che per arrivare ad un nodo va sommato il costo di arrivo con il costo per arrivare al nodo antecedente, guardando ad esempio il nodo 2 uno può pensare che dal nodo 1 al nodo 2 ci sia un costo di 4, tuttavia bisogna sommare a quel 4 il costo 7 (dovuto dal viaggio del nodo 0 al nodo 1) arrivando così ad un costo 11 che è maggiore del costo del



0 = 0		
1 = 7		
2 = 8	2 = 8	
3 =	3 =	3 =
4 = 9	4 = 9	4 = 9
5 =	5 = 13	5 = 13
6 =	6 = 15	6 = 15
7 =	7 =	7 =

Al passo 3 segniamo i rimanenti e scegliamo sempre il costo minore che questa volta è il 4. Da notare che non segniamo il costo dei nodi che non possiamo raggiungere e che segniamo il costo minimo (sommando sempre il percorso) per raggiungere i nodi rimanenti. Continuare fino alla fine del grafo.



0 = 0						
1 = 7						
2 = 8	2 = 8					
3 =	3 =	3 =	3 =	3 = 19	3 = 19	3 = 19
4 = 9	4 = 9	4 = 9				
5 =	5 = 13	5 = 13	5 = 13			
6 =	6 = 15	6 = 15	6 = 14	6 = 14		
7 =	7 =	7 =	7 =	7 = 16	7 = 16	

Nodo 1 [0,1] costo 7

Nodo 2 [0,2] costo 8

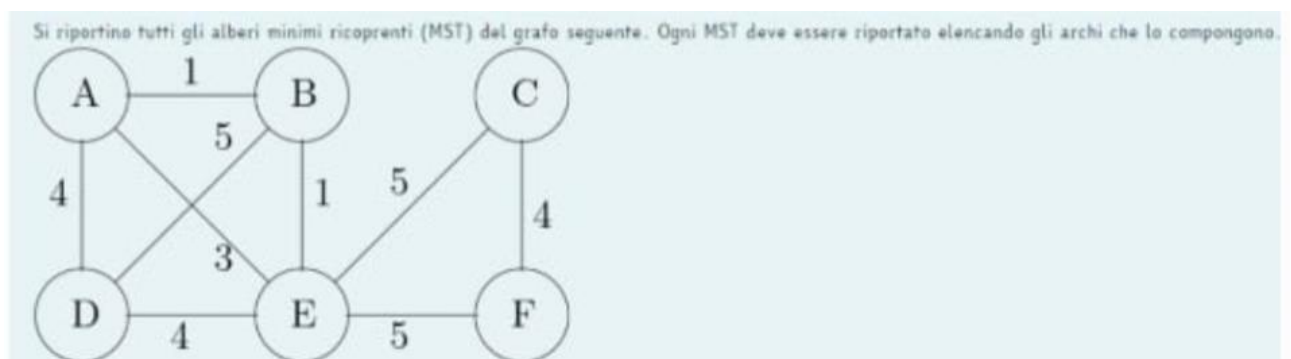
Nodo 3 [0, 1, 5, 3] costo 19

Nodo 4 [0,4] costo 9

Nodo 5 [0,1, 5] costo 13

Nodo 6 [0, 4, 6] costo 14

Nodo 7 [0, 1, 5, 7] costo 16



AB, BE, AD, EC, CF = 15

AB, BE, AD, EF, CF = 15

AB, BE, ED, EC, CF = 15

AB, BE, ED, EF, CF = 15

Visitare i nodi

Si effettui la visita in profondità, a partire dal nodo b, del seguente grafo di otto nodi dato tramite le liste di adiacenza (i nodi adiacenti devono essere considerati nell'ordine in cui appaiono nella lista di adiacenza):

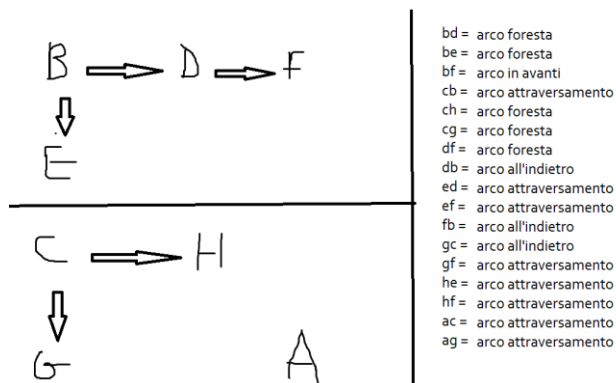
- b: d, e, f
- c: b, h, g
- d: f, b
- e: d, f
- f: b
- g: c, f
- h: e, f
- a: c, g

Si riporti per ogni nodo i tempi di inizio e fine visita (ossia i valori degli attributi *d* ed *f*) e per ogni arco il suo tipo secondo la classificazione degli archi durante una visita in profondità.

Per risolvere questo tipo di esercizi consultare questo [video](#). Ps: grazie al collega che ha realizzato il video.

B: D, E, F	NODI	D	F
C: B, H, G	A	15	16
D: F, B	B	1	8
E: D, F	C	9	14
F: B	D	2	5
G: C, F	E	6	7
H: E, F	F	3	4
A: C, G	G	12	13
	H	10	11

Per specificare gli archi facciamo così:



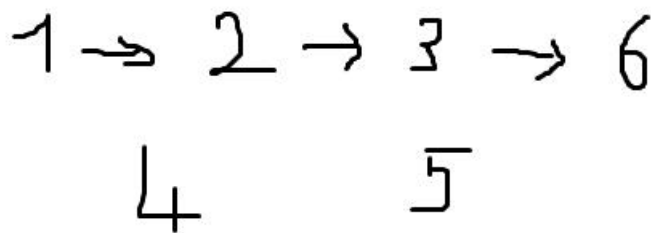
scrivo quei disegnini orrendi guardando i tempi di scorrimento, poi mi scrivo tutte le possibili combinazioni da una parte, se i due nodi sono collegati adiacenti (tipo b->d) dalla freccia è arco foresta, se sono collegati con una cosa in mezzo è arco in avanti (b,f), se sono collegati ma nel senso contrario della freccia (d,b) è arco all'indietro, tutto il resto è arco attraversamento

Durante la visita in profondità di un grafo orientato si registrano i seguenti tempi di *inizio visita* e *fine visita*:

- nodo 1: 1,8
- nodo 2: 2,7
- nodo 3: 3,6
- nodo 4: 9,10
- nodo 5: 11,12
- nodo 6: 4,5

Si classifichino i seguenti archi:

2 → 3	Scegli...	▼	ARCO IN AVANTI
1 → 3	Scegli...	▼	ARCO NELLA FORESTA GENERATA
4 → 2	Scegli...	▼	ARCO IMPOSSIBILE NEL GRAFO
6 → 1	Scegli...	▼	ARCO ALL'INDIETRO
6 → 5	Scegli...	▼	ARCO IMPOSSIBILE NEL GRAFO
1 → 4	Scegli...	▼	ARCO IMPOSSIBILE NEL GRAFO



Partiamo segnando 1, poi disegniamo il nodo 2, poi il nodo 3, poi dopo 3 avremmo il 4 ma vediamo che va da 9 a 10, quindi non è collegato con i precedenti, lo disegniamo da solo, stessa cosa per il nodo 5. Il nodo 6 avendo il 4 è collegato al 3.

PS: le soluzioni che hanno messo in rosso non mi sembrano giuste, tipo 2 → 3 dovrebbe essere arco nella foresta

Si considerino le funzioni hash

$$h_1(k) = k \bmod m, \quad h_2(k) = 1 + (k \bmod m)$$

con cui è definita il doppio hashing:

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m$$

Lo stato della tabella hash ad indirizzamento aperto $T[0..12]$, di dimensione $m = 13$ dopo l'inserimento delle chiavi: 50, 69, 72, 98, 79, 14 nell'ordine dato, risulta essere (indicando con -1 una locazione libera di T):

- ☐ [-1, 79, 98, 14, 69, -1, -1, 72, -1, -1, -1, 50, -1]
- ☐ [-1, 79, -1, -1, 69, 98, -1, 72, -1, 14, -1, 50, -1]
- ☒ [-1, -1, 98, 14, 69, -1, -1, 72, -1, 79, -1, 50, -1]

Per risolvere questo tipo di esercizi consultare prima di tutto questo [link](#).

	h1	h2	
50	11	12	
69	4	5	
72	7	8	
98	7	8	
79	1	2	
14	1	2	

-1	79	98	14	69	-1	-1	72	-1	-1	-1	50	-1
0	1	2	3	4	5	6	7	8	9	10	11	12

Quindi mi sono segnato tutti i risultati usando la funzione, quindi con input 50:

$$h_1 = 50 \bmod 13 = 11$$

$$h_2 = 1 + (50 \bmod 13) = 12$$

quindi una volta ottenuto faccio $11 + 0 * 12 = 11$ e posiziono il 50 nell'indice 11 e così via

Sia $T[0..m-1]$ una tabella hash con capacità $m = 10$ e chiavi intere. Qual è lo stato di T dopo aver inserito le chiavi 88, 12, 2, 22, 33, se si utilizza il metodo di indirizzamento aperto e la funzione hash $h(k) = k \bmod 10$?

Scegli un'alternativa:

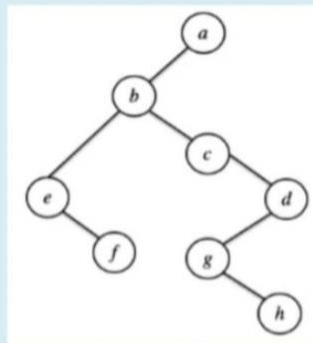
- ☐ a. {nil, nil, 12, 33, 2, 22, nil, nil, 88, nil}
- ☐ b. {nil, nil, 33, 12, 2, 22, nil, nil, 88, nil}
- ☐ c. {nil, nil, 12, 2, 22, 33, nil, nil, 88, nil}
- ☐ d. {nil, nil, 12, 22, 2, 33, nil, nil, 88, nil}

	h	
88	8	
12	2	
2	2	
22	2	
33	3	

		12	2	22	33			88	
0	1	2	3	4	5	6	7	8	9

ALBERO K-ARIO

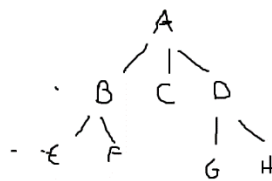
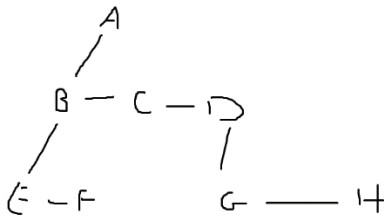
Si consideri la seguente rappresentazione di un albero k -ario con un albero binario, corrispondente alla sua memorizzazione mediante puntatori child/sibling:



Quali delle seguenti affermazioni è corretta?

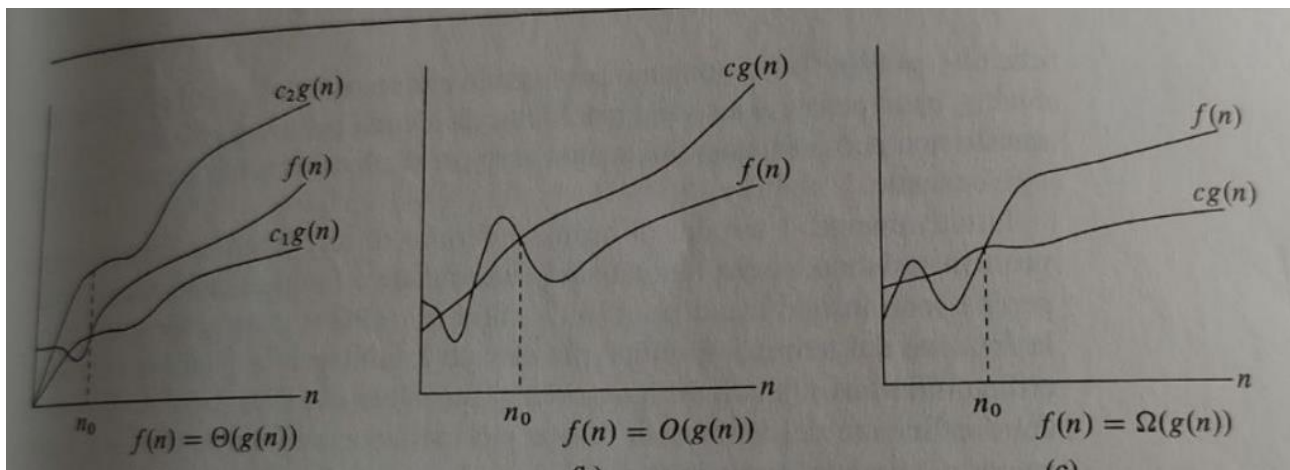
Scegli una o più alternative:

- ☐ a. l'albero ha grado 3 e l'insieme delle sue foglie è $\{c, e, f, g, h\}$;
- ☐ b. l'albero ha grado 3 e i nodi $\{a, b, c, d\}$ sono sullo stesso ramo;
- ☐ c. L'albero ha altezza 5 e l'insieme delle sue foglie è $\{f, h\}$.
- ☐ d. l'albero ha altezza 2 e i nodi $\{a, d, h\}$ formano un ramo;
- ☐ e. l'albero ha grado 2 e i nodi $\{a, b, f\}$ formano un ramo;



Trasformo in albero e vedo che ha grado 3 e altezza 2, inoltre a,d,h formano un ramo quindi le risposte giuste sono **B,D**

ORDINI DI GRANDEZZA



Si valuti con sì/no le seguenti affermazioni:

$$800n + n^2 + n \log n \in O(n \log n)$$

$$\frac{n^2 + 6n + 9}{n+1} \in \Theta(n^2)$$

$$\frac{n^2 + 6n + 9}{n+1} \in \Theta(n)$$

$$3^{n-2} + 2^{n+5} \in \Omega(3^n)$$

Nella prima notiamo che n^2 è il termine dominante, quindi è compreso in $O(n^2)$ quindi la prima è falsa.

Nella seconda n^2 diviene n per il denominatore, quindi appartiene a $\Theta(n)$ quindi anche la seconda è falsa.

Di conseguenza la terza è vera.

Nella quarta scomponendo il primo termine si ottiene $3^n - 3^2$, quindi effettivamente appartiene a $\Omega(3^n)$

Si dica quali dei seguenti asserti equivale a $g(n) \in \Theta(f(n))$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = l > 0 \wedge l < \infty$$

Scegli... ▼

$$g(n) \in O(f(n)) \wedge f(n) \in \Omega(g(n))$$

Scegli... ▼

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

Scegli... ▼

$$g(n) \in O(f(n)) \wedge g(n) \in \Omega(f(n))$$

Scegli... ▼

1. vero 2. falso 3. falso 4. Vero

Siano $f(n), g(n)$ funzioni asintoticamente positive; si dica quali delle seguenti

se $f(n) \in \Theta(g(n))$ allora $g(n) \in \Theta(f(n))$

se $f(n) \in O(g(n))$ allora $2^{f(n)} \in O(2^{g(n)})$

$\min\{f(n), g(n)\} \in \Theta(f(n) + g(n))$

$f(n) \in \Theta(f(n)/2)$

1. Si 2. Si 3. no 4. No

Si considerino le funzioni $f(n) = n$ e $g(n) = n^{1+\sin n}$. Si valuti con si/no le seguenti affermazioni:

$f(n) \notin \Omega(g(n))$ Scegli...

$f(n) \notin O(g(n))$ Scegli...

$g(n) \in O(f(n))$ Scegli...

$f(n) \in \Omega(g(n))$ Scegli...

1. Si 2. Si 3. No 4. No

Sia $f(n)$ una funzione non decrescente e positiva per qualunque n .

$f(n) \in O(g(n))$ implica $f(n) \notin \Omega(g(n))$? Scegli...

$f(n) \in O(g(n))$ implica $g(n) \in \Omega(f(n))$? Scegli...

$f(n) \in O(g(n))$ implica $f(n) \notin \Theta(g(n))$? Scegli...

$f(n) \notin O(g(n))$ implica $f(n) \notin \Theta(g(n))$? Scegli...

1. si 2. si 3. no 4. si

Complessità temporale

21:01



Si analizzi la complessità temporale in funzione di n del seguente algoritmo:

```
ALG( $n$ )
  if  $n \leq 1$  then return 5
  else
     $m \leftarrow 0$ 
    for  $i \leftarrow 1$  to 4 do
       $m \leftarrow m + 2 \cdot \text{ALG}(\lfloor n/2 \rfloor)$ 
    for  $i \leftarrow 1$  to  $n$  do
      for  $j \leftarrow 1$  to  $n$  do
         $m \leftarrow m + 1$ 
    return  $m$ 
```

La sua funzione tempo soddisfa la relazione di ricorrenza

- ☐ $T(n) = 8T(n/2) + 2n^2$
- ☐ $T(n) = 8T(n/2) + n^2$
- ☒ $T(n) = 4T(n/2) + n^2$ ✓

che dopo $k \leq \log_2 n$ svolgimenti ha la forma

- ☐ $2^k T(n/2^k) + 2kn^2$
- ☒ $4^k T(n/2^k) + kn^2$ ✓
- ☐ $8^k T(n/2^k) + 2kn^2$

da cui risulta che $T(n)$ ha complessità

- ☒ $\Theta(n^2 \log n)$ ✓
- ☐ $\Theta(n \log^2 n)$
- ☐ $\Theta(n^3 \log n^2)$



Per capire la relazione di ricorrenza dobbiamo analizzare il codice e vedere se (e quante volte) viene richiamato l'algoritmo ricorsivamente e la grandezza dei for.

In questo esercizio notiamo che nel primo for che va da 1 a 4 viene fatta una chiamata ricorsiva, siccome viene svolta 4 volte allora abbiamo un tempo $4T$, successivamente dobbiamo sommarli 2 volte n , poiché ci sono due for annidati che vanno fino ad n , perciò è n^2 .

Per il secondo step andiamo a esclusione, quindi se fa k svolgimenti allora sarà $4^k(n/2)$ sommandogli 4 volte n^2 .

La terza parte non l'ho capita bene dio...



Si consideri l'algoritmo:

```
ALGO( $n$ )
  Pre:  $n \geq 0$ 
  if  $n = 0$  then
    return 0
  else
     $j \leftarrow 1$ 
    for  $i \leftarrow 1$  to  $n$  do
       $j \leftarrow n \cdot i$ 
    return  $\text{ALGO}(n-1) + \text{ALGO}(n-1)$ 
```

La sua funzione tempo in termini di n soddisfa la relazione di ricorrenza:

- ☐ $T(n) = T(n-1) + n$
- ☒ $T(n) = 2T(n-1) + n$ ✓
- ☐ $T(n) = 2T(n-1) + 1$

che dopo k svolgimenti ha la forma

• a:

$$2^k T(n/2^k) + \sum_{i=0}^k 2^i$$

• b:

$$T(n-k) + \sum_{i=0}^{k-1} (n-i)$$

• c:

$$2^k T(n-k) + \sum_{i=0}^{k-1} 2^i (n-i)$$

☐ a

☐ b

☒ c ✓

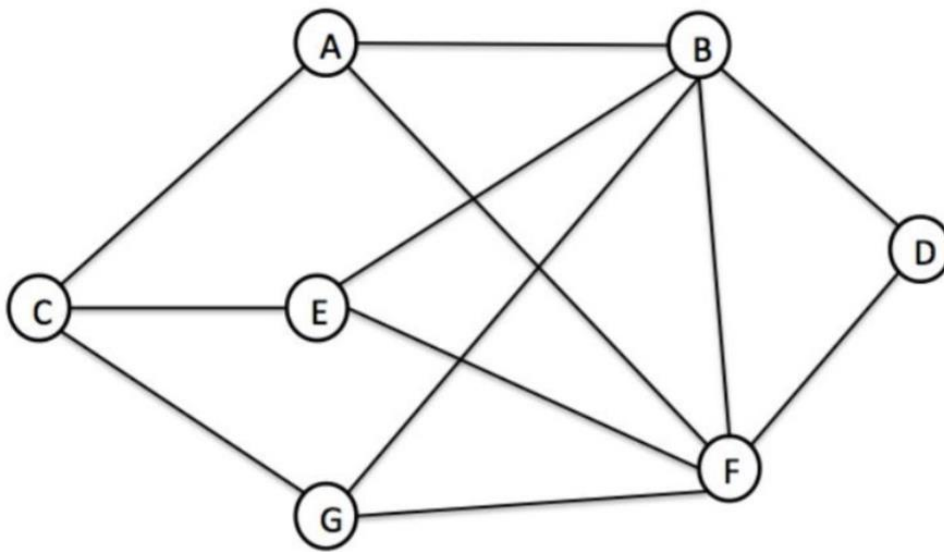
da cui risulta che $T(n)$ ha complessità

- ☐ $\Theta(n^2)$
- ☐ $\Theta(n!)$
- ☒ $\Theta(2^n)$ ✓

Qua vediamo che l'algoritmo viene richiamato 2 volte, quindi $2T$ a cui gli fa sommato una volta n visto che il for scorre fino ad n .

ES4

Un grafo $G = (V, E)$ si dice *bipartito* se esiste una partizione di V in due sottoinsiemi V_1 e V_2 tale che nessun arco $(u, v) \in E$ abbia i vertici u, v contenuti nella stessa parte, ossia per ogni $(u, v) \in E$ o $u \in V_1$ e $v \in V_2$, oppure $u \in V_2$ e $v \in V_1$. Si decida se il seguente grafo è bipartito oppure no, fornendo una partizione V_1, V_2 di V nel caso affermativo; si spieghi perché nessuna partizione di V soddisfa la proprietà succitata nel caso negativo.



Un grafo si dice bipartito se al suo interno non sono presenti cicli di lunghezza dispari, tipo un triangolo (c'è in questo disegno quindi non è bipartito).